

MACHINE LEARNING PROJECT PRESENTATION

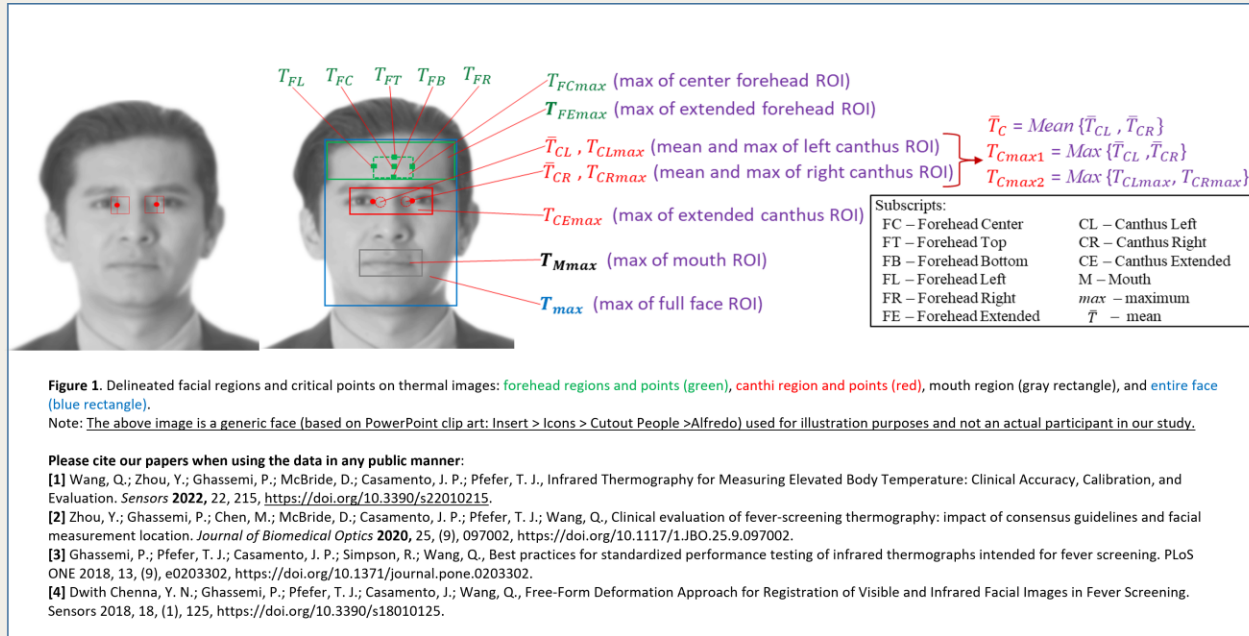
Presented By:

Shagufta Shaheen (477654)
Kacper Geisshirt (443171)

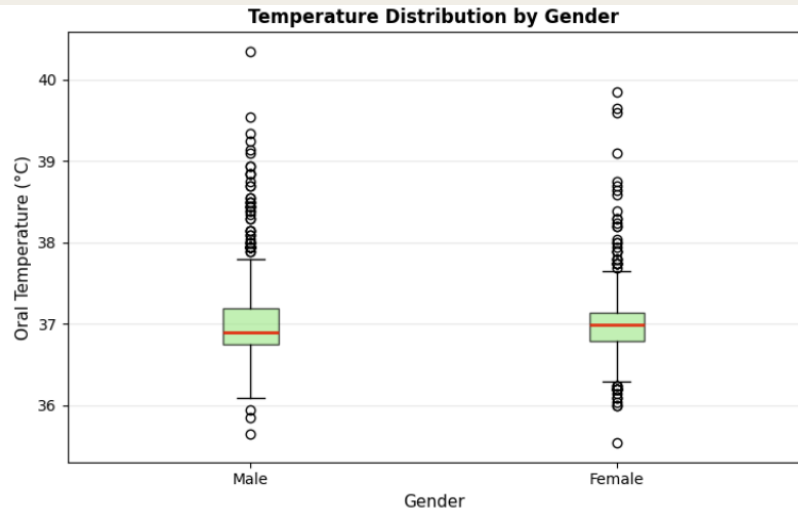
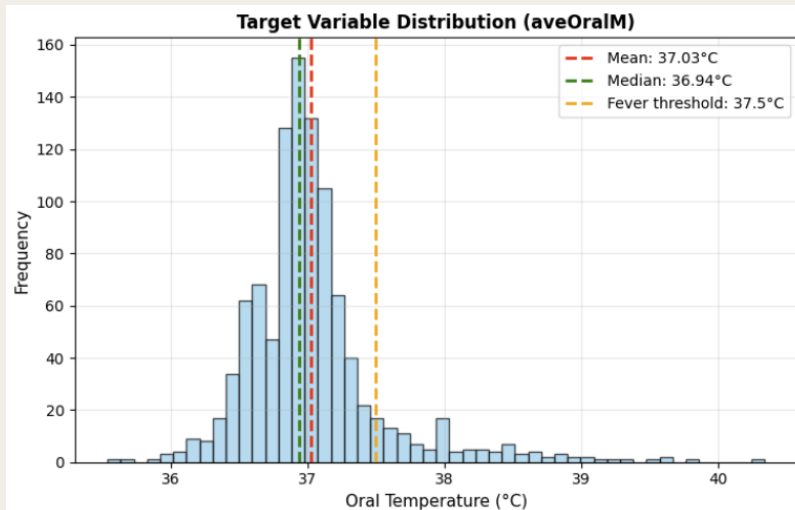
Under the guidance of:

Dr. Paweł Sakowski,

REGRESSOR FOR ORAL TEMPERATURE



DATASET



Target: aveOralM – oral temperature measured by a thermometer.

MODELING APPROACH

Models used:

- Random Forest
- XGBoost
- Neural Network

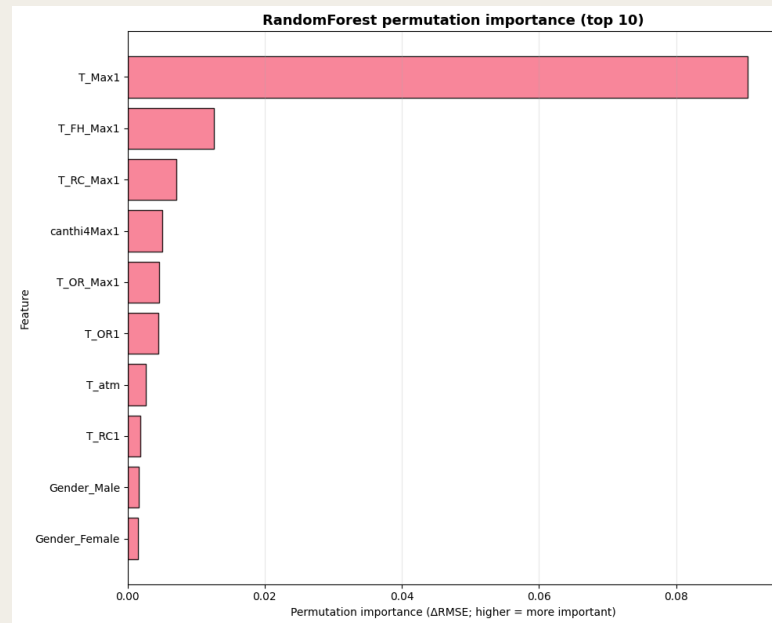
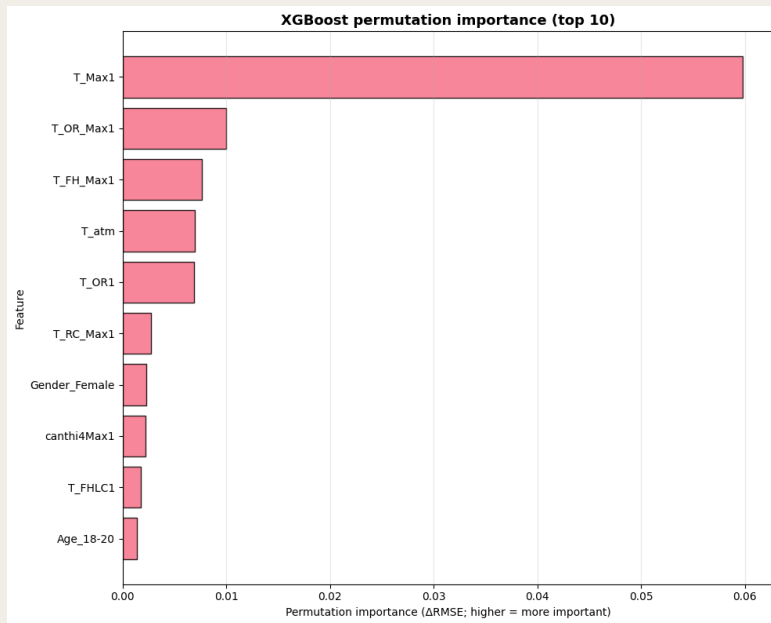
Tuning was done with a
Randomized Search CV.

```
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
])

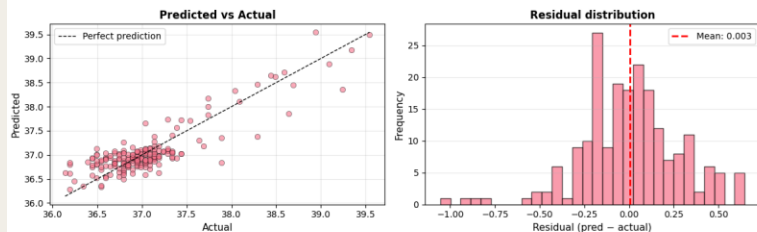
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features),
    ],
    remainder="drop",
    verbose_feature_names_out=False,
)
```

FEATURE PERMUTATION IMPORTANCE

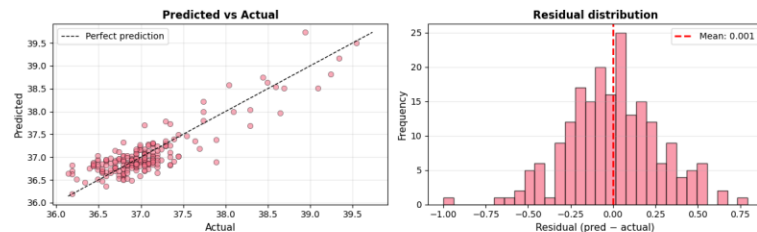


DIAGNOSTICS

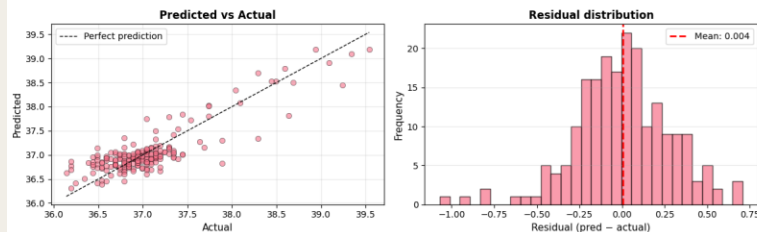
Diagnostics — XGBoost (tuned)



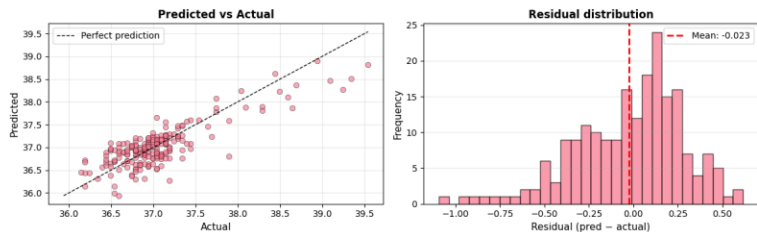
Diagnostics — MLP (tuned)



Diagnostics — RandomForest (tuned)

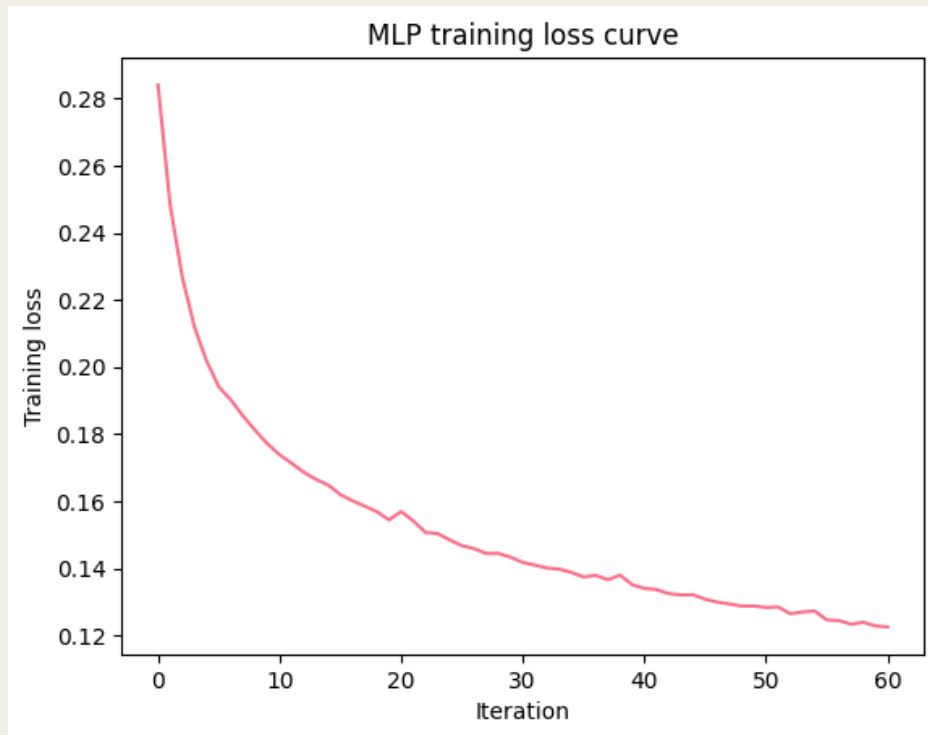


Diagnostics — LinearRegression (baseline)



LOSS CURVE

- Early Stopping
- Max_iteration = 1000
- Actually run only 61



RESULTS

	CvRMSEmean	CvRMSEstd	Relative gap cv	Train RMSE	Test RMSE	Test MAE	Test R ²	Relative gap test
	0.295264	0.027040	<u>0.118030</u>	0.260414	<u>0.272148</u>	0.211304	0.760891	<u>0.043115</u>
XGB	<u>0.290753</u>	0.023480	0.307179	<u>0.201440</u>	0.276135	<u>0.209293</u>	0.753833	0.270501
	0.295965	<u>0.021497</u>	0.279726	0.213176	0.279805	0.212283	0.747246	0.238126
LR	0.327963	0.028691	0.096198	0.296414	0.305940	0.241041	0.697823	0.031138

CLASSIFIERS FOR LOW-ALTITUDE DISASTER IMAGERY



1.DATASET

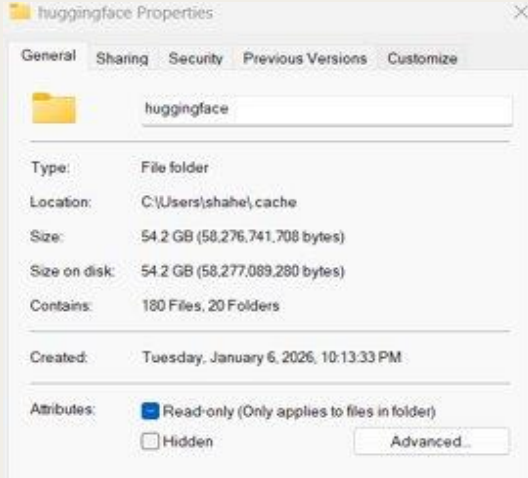
1. Dataset Overview :

- Data Name: **LADI-v2-dataset**
- Dataset Size: **~55 GB**
- Source: MITLL/LADI-v2-datase
- <https://huggingface.co/datasets/MITLL/LADI-v2-dataset>

2. Why this Dataset? :

- Aerial disaster images collected by **Civil Air Patrol (CAP)**
- Images are **geotagged** and carefully **multi-labelannotated**
- Labels verified by **trained FEMA volunteers** usingmajority voting
- Contains **~10,000 images** (train, validation, test split)
- High-quality labels suitable for **deep learning models**
- Supports **automatic damage detection** and emergencyresponse

DATA ACQUISITION & ANALYSIS PLAN



- Dataset downloaded from **Hugging Face** using the datasets library
- Data is downloaded **once** and stored in a **global cache (~55 GB)**
- Cached data is reused in every run (no repeated downloads)
- Basic preprocessing applied before model training
- Prepared data used for analysis and deep learning experiments

DATA DESCRIPTION

v2a

bridges_any
buildings_any
buildings_affected_or_greater
buildings_minor_or_greater
debris_any
flooding_any
flooding_structures
roads_any
roads_damage
trees_any
trees_damage
water_any

- Each image is of size 1800 × 1200 pixels
- Dataset split:
 - **Training set:** ~8,000 images
 - **Validation set:** ~1,000 images
 - **Test set:** ~1,000 images
- Training and validation data are from **CAP disaster events (2015–2022)**
- Test data is from **2023 disaster events**
- Test set introduces a **distribution shift** (new event types and locations)
- Dataset includes **12 damage classification levels**

```
[V]: da
[[{"DatasetDir": {"train": Dataset({
    features: ['image', 'bridges_any', 'buildings_any', 'buildings_affected_or_greater', 'buildings_minor_or_greater', 'debris_any', 'flooding_any',
    'flooding_structures', 'roads_any', 'roads_damage', 'trees_any', 'trees_damage', 'water_any'],
    num_rows: 8000
}),
    "validation": Dataset({
    features: ['image', 'bridges_any', 'buildings_any', 'buildings_affected_or_greater', 'buildings_minor_or_greater', 'debris_any', 'flooding_any',
    'flooding_structures', 'roads_any', 'roads_damage', 'trees_any', 'trees_damage', 'water_any'],
    num_rows: 1000
}),
    "test": Dataset({
    features: ['image', 'bridges_any', 'buildings_any', 'buildings_affected_or_greater', 'buildings_minor_or_greater', 'debris_any', 'flooding_any',
    'flooding_structures', 'roads_any', 'roads_damage', 'trees_any', 'trees_damage', 'water_any'],
    num_rows: 1000
})
}]]
```

MODEL TRAINING STRATEGIES

1. Training a ConvNet from Scratch
2. Data Augmentation
3. Using a Pretrained Model (ResNetV2)
4. Feature Extraction with a Pretrained Model
5. Fine-Tuning a Pretrained Model

1. TRAINING A CONVNET FROM SCRATCH

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 180, 180, 3)	0
rescaling (Rescaling)	(None, 180, 180, 3)	0
conv2d (Conv2D)	(None, 178, 178, 32)	896
max_pooling2d (MaxPooling2D)	(None, 89, 89, 32)	0
conv2d_1 (Conv2D)	(None, 87, 87, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 43, 43, 64)	0
conv2d_2 (Conv2D)	(None, 41, 41, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 20, 20, 128)	0
conv2d_3 (Conv2D)	(None, 18, 18, 256)	295,168
max_pooling2d_3 (MaxPooling2D)	(None, 9, 9, 256)	0
conv2d_4 (Conv2D)	(None, 7, 7, 256)	590,080
flatten (Flatten)	(None, 12544)	0
dense (Dense)	(None, 12)	150,540

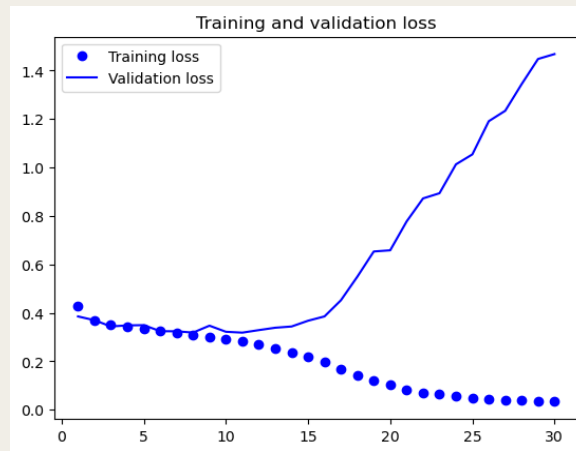
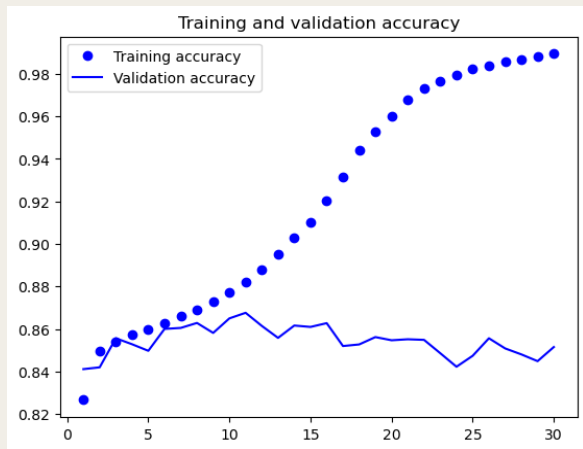
Total params: 1,129,036 (4.31 MB)
Trainable params: 1,129,036 (4.31 MB)
Non-trainable params: 0 (0.00 B)

- Input images resized to **180 × 180 × 3** and normalized
- Custom CNN with **5 Conv2D layers**
- **MaxPooling** for feature extraction
- Filters increase progressively: **32 → 64 → 128 → 256**
- Features flattened and classified using a **Dense layer** (12 outputs)
- Total trainable parameters: **~1.13M**

Training Setup

- Optimizer: **RMSprop**
- Loss: **Binary Crossentropy**
- Metric: **Binary Accuracy**
- Trained for **30 epochs** with validation monitoring
- **Best model saved** using ModelCheckpoint

1. TRAINING A CONVNET FROM SCRATCH CONT.



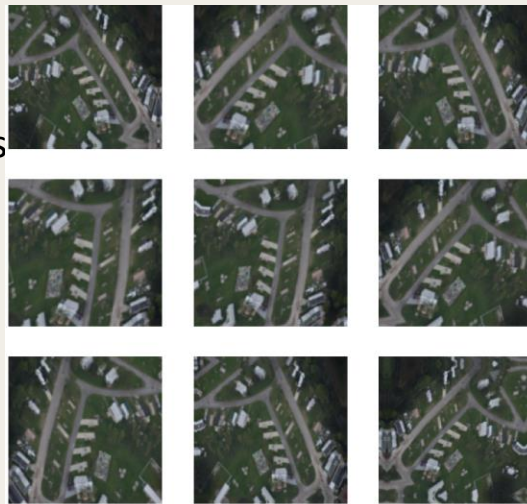
33/33 ————— **138s** 4s/step - binary_accuracy: 0.8724 - loss: 0.2955
Test accuracy: 0.872

2. DATA AUGMENTATION

- Applied dynamically during training using Keras
- Techniques: **horizontal flip**, **rotation (0.1)**, **zoom (0.2)**
- Increases data diversity without collecting new samples
- Reduces overfitting and improves generalization
- Especially useful for **12-label multi-label classification**

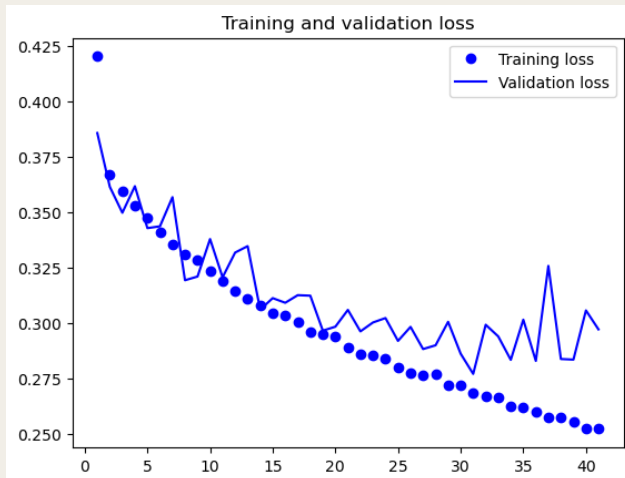
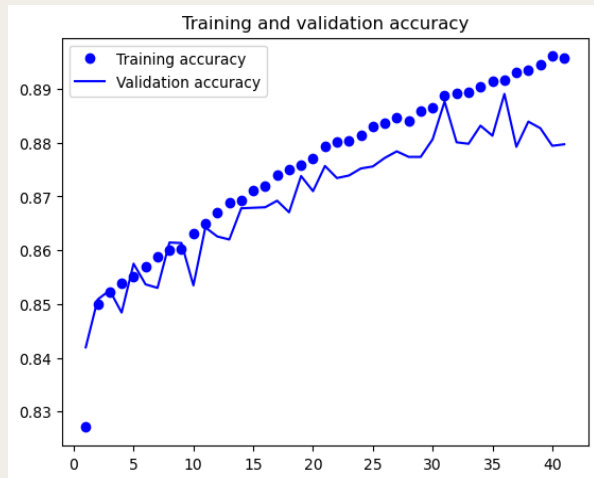
Training Strategy

- **Dropout (0.5)** used to reduce overfitting
- **EarlyStopping** applied based on validation loss
- **ModelCheckpoint** saves the best model
- Trained for up to **100 epochs** with validation monitoring



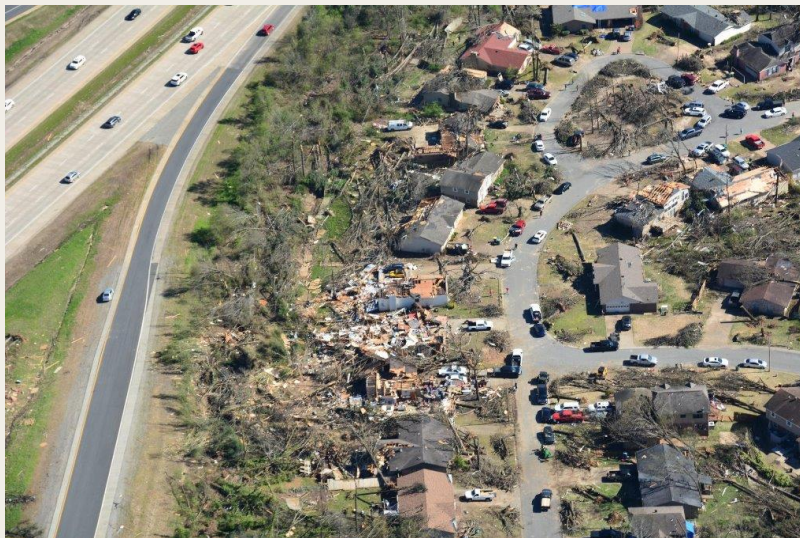
```
data_augmentation = keras.Sequential([  
    layers.RandomFlip("horizontal"),  
    layers.RandomRotation(0.1),  
    layers.RandomZoom(0.2),  
])
```


2. DATA AUGMENTATION (CONT.)



33/33 ————— 126s 4s/step - binary_accuracy: 0.8684 - loss: 0.3052
Test accuracy: 0.868

PREDICTION RESULT (AFTER DATA AUGMENTATION)

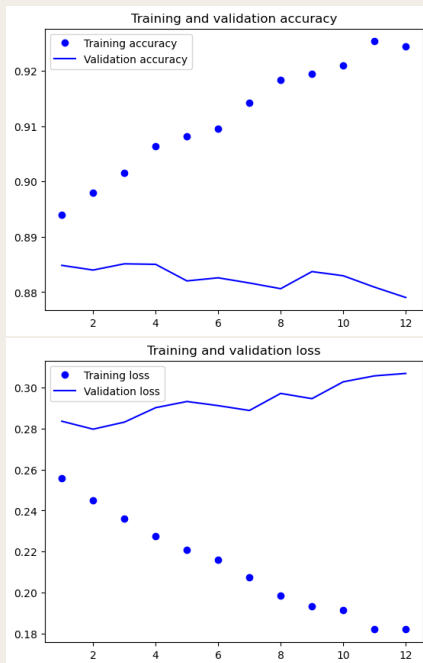


Label	True	Prob (%)	Pred
bridges_any	False	10.60 %	False
buildings_any	True	99.00 %	True
buildings_affected_or_greater	True	51.06 %	True
buildings_minior_or_greater	True	18.53 %	False
debris_any	True	27.16 %	False
flooding_any	False	4.73 %	False
flooding_structures	False	1.04 %	False
roads_any	True	98.96 %	True
roads_damage	False	3.38 %	False
trees_any	True	95.37 %	True
trees_damage	True	38.79 %	False
water_any	False	38.09 %	False

3. PRETRAINED MODEL – RESNETV2

- 50-layer deep residual CNN pretrained on ImageNet
- Residual connections enable stable and deep feature learning
- Implementation Workflow:
 - Fast feature extraction without data augmentation
 - Fine-tuning of the pretrained model
- Achieved faster training and better performance than training from scratch

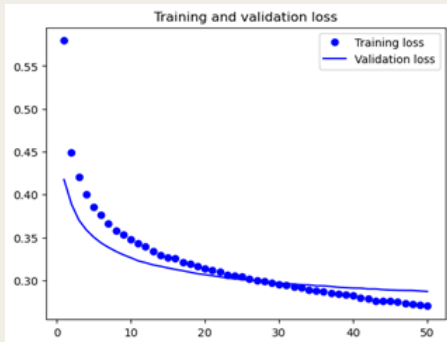
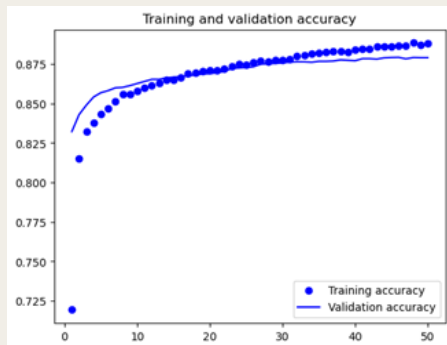
3(I). FEATURE EXTRACTION WITHOUT DATA AUGMENTATION



- **Efficient Feature Logging:** Ran the convolutional base once over the entire dataset and stored the output as NumPy arrays on disk.
- **Standalone Classification:** Used the recorded features as direct input for a separate, densely connected classifier.
- **Cost-Effective Execution:** Significantly reduced computational cost and time by avoiding repeated passes through the heavy convolutional layers.
- **Performance Speed:** This approach is much faster than end-to-end training because the most expensive part of the pipeline is only run once per image.
- **Augmentation Limitation:** Since features are pre-calculated and static on disk, this specific technique does not allow for real-time data augmentation

33/33 ————— 0s 4ms/step - binary_accuracy: 0.8904 - loss: 0.2823
Test accuracy: 0.890

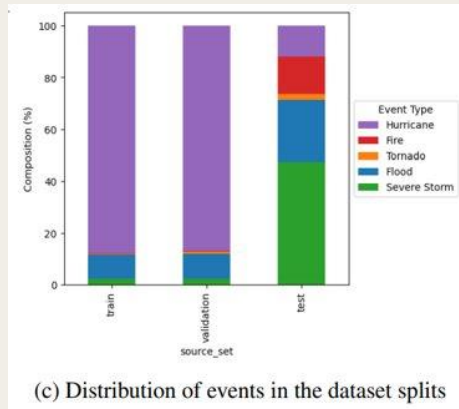
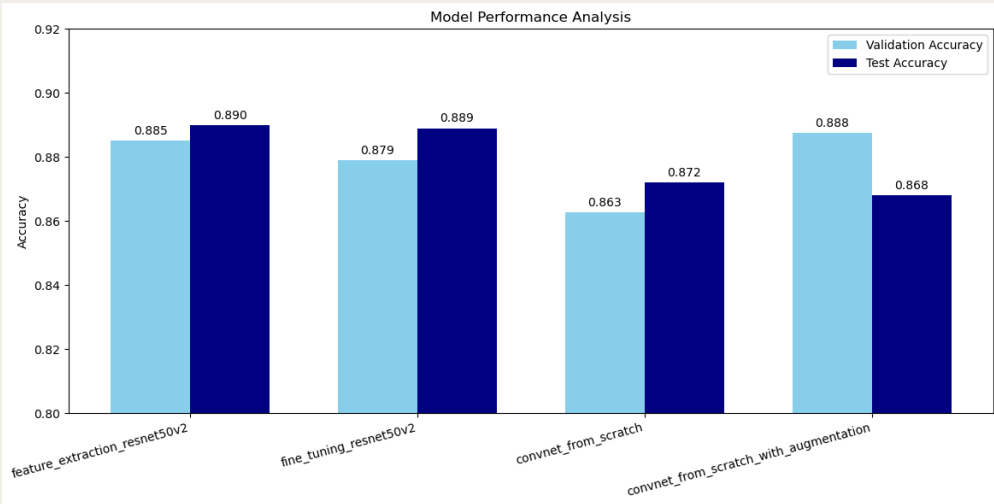
3(II). FINE-TUNING A PRETRAINED MODEL



- Freezed the base network and added our custom classifier on top of the already-trained base
- Initially trained only the newly added part to keep the base weights stable
- Unfreezed every layer except the last 3 of the ResNet base network
- Jointly trained these specific base layers along with the custom network we added
- Adjusted the high-level features of the pretrained model to better fit our specific disaster images

33/33 — 0s 3ms/step - binary_accuracy: 0.8892 - loss: 0.2723
Test accuracy: 0.889

PERFORMANCE ANALYSIS



(c) Distribution of events in the dataset splits

- Relatively low performance is likely due to the limited number of fire events in the training data
- Performance differences arise from the visual complexity of smoke and fire compared to structural damage

An aerial photograph showing a large industrial facility, possibly a refinery or chemical plant, completely surrounded by floodwater. The facility includes several large, cylindrical storage tanks, a few rectangular buildings, and some smaller structures. A multi-lane highway is visible in the upper left corner, partially submerged. The water is a murky, brownish-grey color. The text "THANK YOU FOR YOUR ATTENTION" is overlaid in large, white, sans-serif capital letters across the upper half of the image.

THANK YOU FOR YOUR ATTENTION

Do you have any questions?

REFERENCES & RESOURCES

- **He, K., Zhang, X., Ren, S., & Sun, J. (2016)**
Deep Residual Learning for Image Recognition
IEEE Conference on Computer Vision and Pattern Recognition (CVPR)
<https://arxiv.org/abs/1512.03385>
- **Keras Documentation – Convolutional Neural Networks**
https://keras.io/guides/transfer_learning/
https://keras.io/api/layers/convolution_layers/
- **TensorFlow & Keras API Documentation**
https://www.tensorflow.org/api_docs
<https://keras.io/api/>
- **ResNetV2 Architecture**
<https://keras.io/api/applications/resnet/#resnetv2-function>
- **Google Big Transfer (BiT) Models**
Kolesnikov et al., *Big Transfer (BiT): General Visual Representation Learning*
<https://arxiv.org/abs/1912.11370>
https://github.com/google-research/big_transfer
<https://huggingface.co/google/bit-50>
- **Binary Cross-Entropy Loss**
https://keras.io/api/losses/probabilistic_losses/#binary_crossentropy-class
- **RMSprop Optimizer**
<https://keras.io/api/optimizers/rmsprop/>